

Software Básico

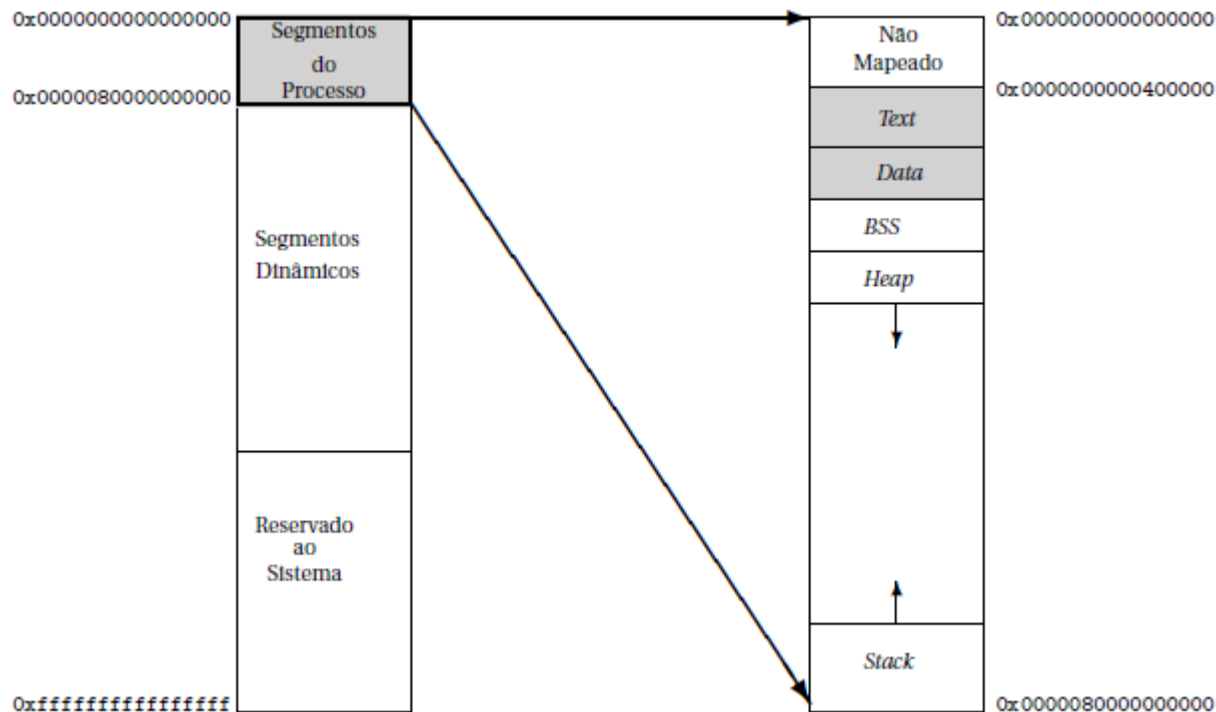


**INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
NORTE DE MINAS GERAIS**

fppt.com

Tradução e execução de programas

- Segmento de processo é dividido em partes, ou seções, chamadas seção **text**, **data**, **bss**, **heap** e **stack**.



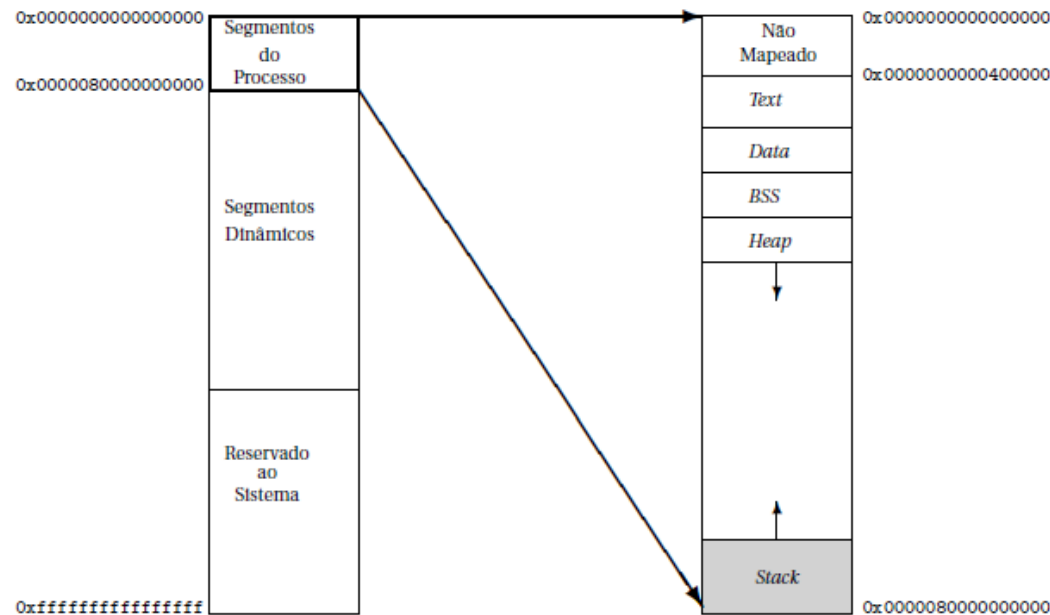
Modelo de procedimento

- A seção da pilha (*stack*) armazena informações sobre chamadas de procedimento:
 - parâmetros,
 - informações sobre contexto e
 - variáveis locais.
- Em linguagens antigas as variáveis eram definidas como globais.



Modelo de procedimento

- A seção da pilha (*stack*)



- O intervalo entre o endereço virtual `0x0000080000000000` e um limite variável dado pelo valor atual do registrador `%rsp`.



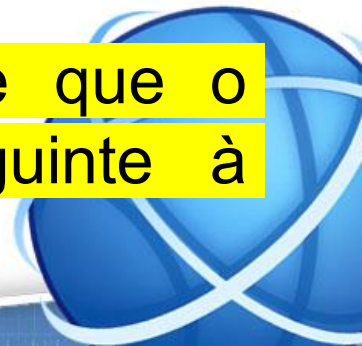
Modelo de procedimento

- Informações armazenadas nas chamadas de procedimento:
 - Escopo de variáveis:
 - Uma variável declarada fora dos procedimentos é uma **variável global**;
 - **Variável** declarada dentro de um procedimento (**local ou parâmetro**) só é visível dentro do procedimento;
 - As variáveis de um procedimento são **alocadas** quando o **procedimento é chamado** e **desalocadas** quando da **saída do procedimento**;



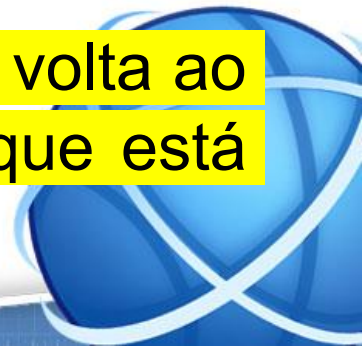
Modelo de procedimento

- Informações armazenadas nas chamadas de procedimento:
 - Informações de Contexto
 - As informações de contexto que devem ser salvas são duas:
 - a que organiza a sequência de chamada e retorno de procedimentos, e
 - a que permite acesso às variáveis de um procedimento.
 - Na saída do procedimento, sabe-se que o fluxo deve retornar à instrução seguinte à chamada.



Modelo de procedimento

- Informações armazenadas nas chamadas de procedimento:
 - Informações de Contexto
 - Quando se faz a **chamada de procedimento**, sabe-se que o fluxo é **desviado para o procedimento**.
 - Antes de desviar o fluxo para o procedimento, armazena-se na pilha o endereço da instrução seguinte.
 - » Para retornar do procedimento de volta ao chamador, utiliza-se o endereço que está disponível no topo da pilha.



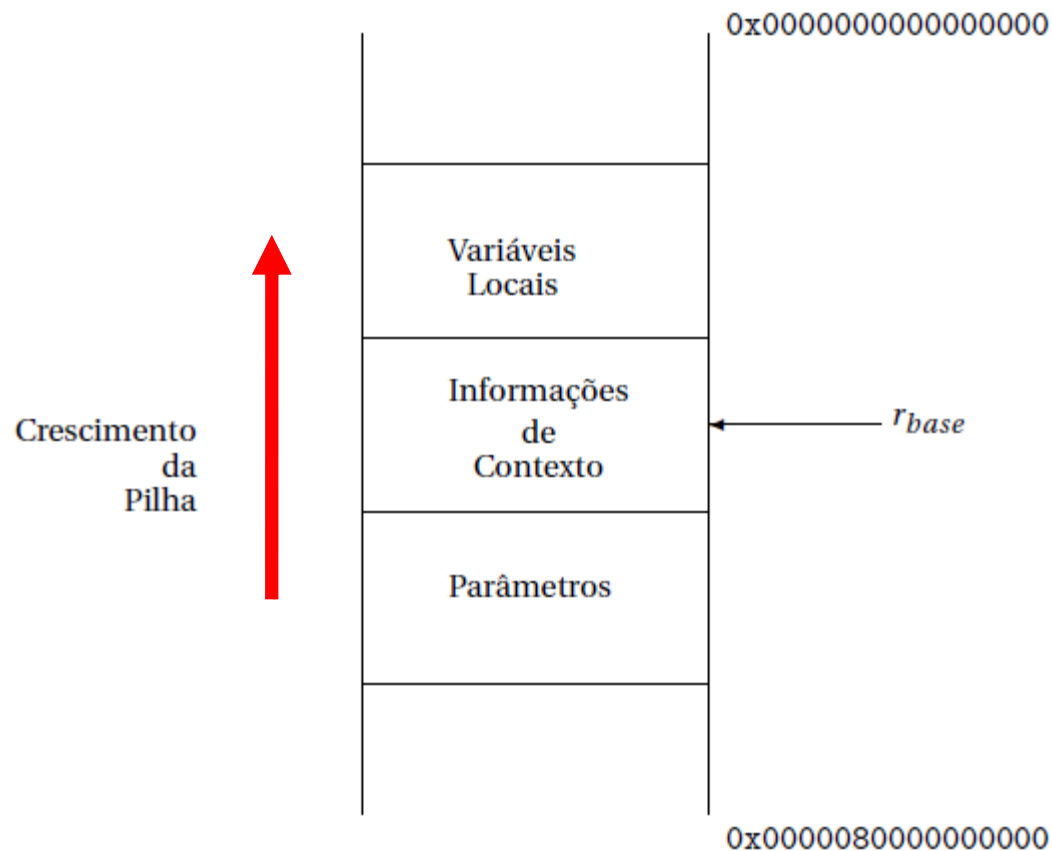
Modelo de procedimento

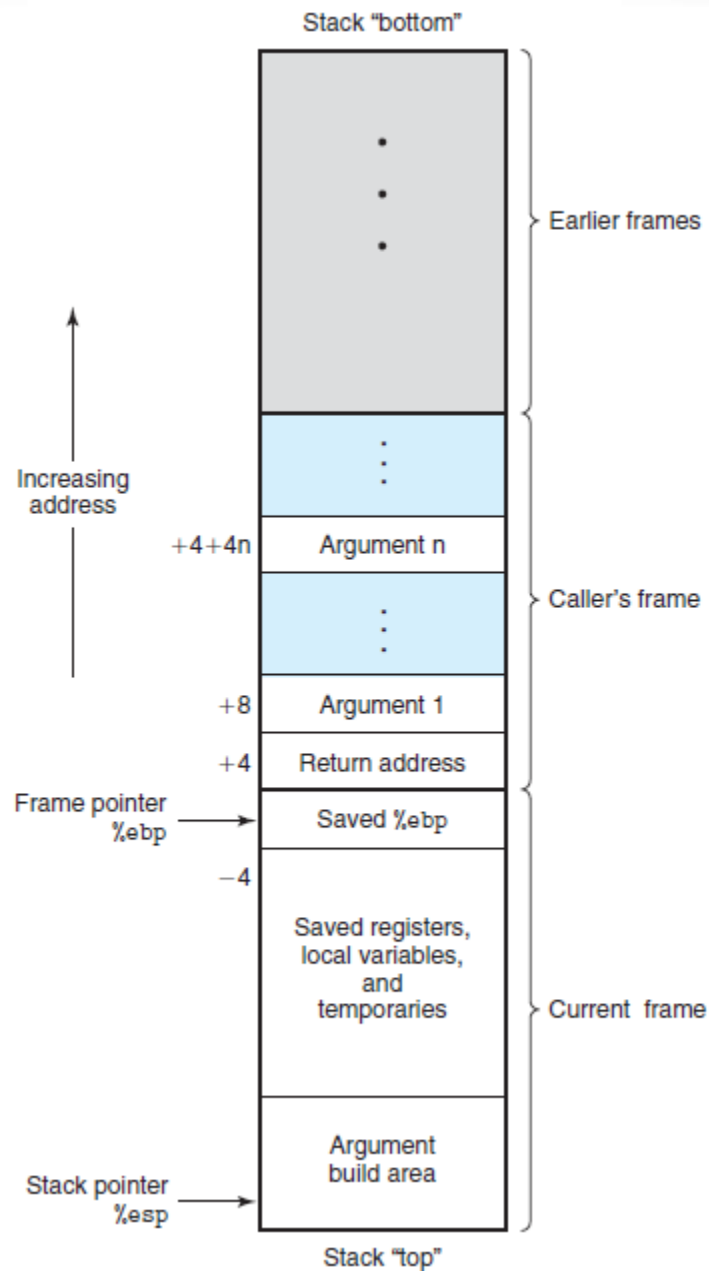
- **Registro de Ativação:**
 - **Registro de Ativação** é o termo adotado para indicar a **estrutura** criada em **tempo de execução** para armazenar:
 - **Variáveis de um procedimento:**
 - variáveis locais;
 - parâmetros;
 - **Informações de contexto:**
 - endereço de retorno;
 - registrador de base.



Modelo de procedimento

- Registro de Ativação:





Modelo de procedimento

Por convenção, os registradores %eax, %edx e %ecx são classificados como **registradores salvos pelo chamador** (*caller-save*).

Quando o procedimento **Q** é chamado por **P**, ele pode sobrescrever esses registradores sem destruir quaisquer dados necessários por **P**.

Por outro lado, os registradores %ebx, %esi e %edi são classificados como **registradores salvos pelo chamado** (*callee-save*).



Modelo de procedimento

Isso significa que Q deve salvar os valores de quaisquer desses registradores na pilha antes de sobrescrevê-los, e restaurá-los antes de retornar, **porque P** (ou algum procedimento de nível superior) **pode precisar desses valores para seus cálculos futuros.**

Além disso, os registradores %ebp e %esp devem ser mantidos de acordo com as convenções aqui descritas.



Modelo de procedimiento

```
int swap_add(int *xp, int *yp)
{
    int x = *xp;
    int y = *yp;

    *xp = y;
    *yp = x;
    return x + y;
}

int caller()
{
    int arg1 = 534;
    int arg2 = 1057;
    int sum = swap_add(&arg1, &arg2);
    int diff = arg1 - arg2;

    return sum * diff;
}
```



Modelo de procedimiento

```
1 caller:
2   pushl   %ebp
3   movl    %esp, %ebp
4   subl    $24, %esp
5   movl    $534, -4(%ebp)
6   movl    $1057, -8(%ebp)
7   leal    -8(%ebp), %eax
8   movl    %eax, 4(%esp)
9   leal    -4(%ebp), %eax
10  movl    %eax, (%esp)
11  call     swap_add
12  movl    -4(%ebp), %edx
13  subl    -8(%ebp), %edx
14  imull    %edx, %eax
15  leave
16  ret
```

```
1 swap_add:
2   pushl   %ebp
3   movl    %esp, %ebp
4   pushl    %ebx
5   movl    8(%ebp), %edx
6   movl    12(%ebp), %ecx
7   movl    (%edx), %ebx
8   movl    (%ecx), %eax
9   movl    %eax, (%edx)
10  movl    %ebx, (%ecx)
11  addl    %ebx, %eax
12  popl    %ebx
13  popl    %ebp
14  ret
```



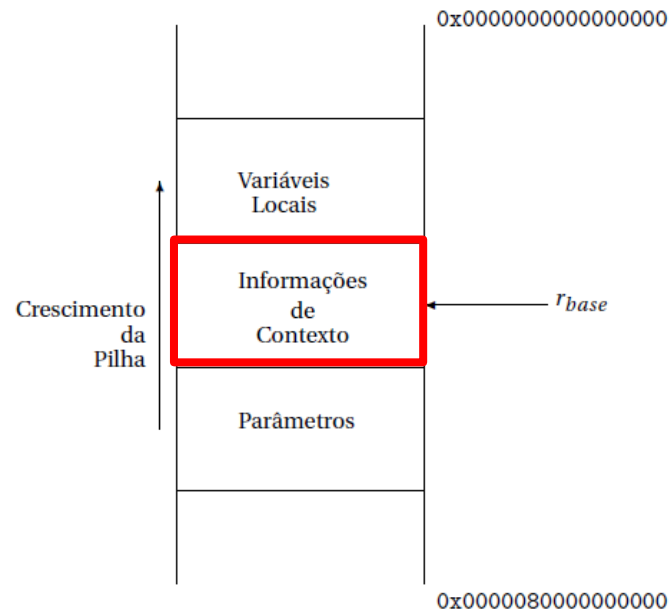
Linguagem Assembly

- **Modelo de implementação de procedimento:**
 - Alocação de registro de ativação:
 - **Sem Parâmetros e sem Variáveis Locais;**
 - **Sem Parâmetros e com Variáveis Locais;**
 - **Com Parâmetros e com Variáveis Locais;**
 - **Parâmetros por Referência e com Variáveis Locais;**
 - **Chamadas Recursivas.**



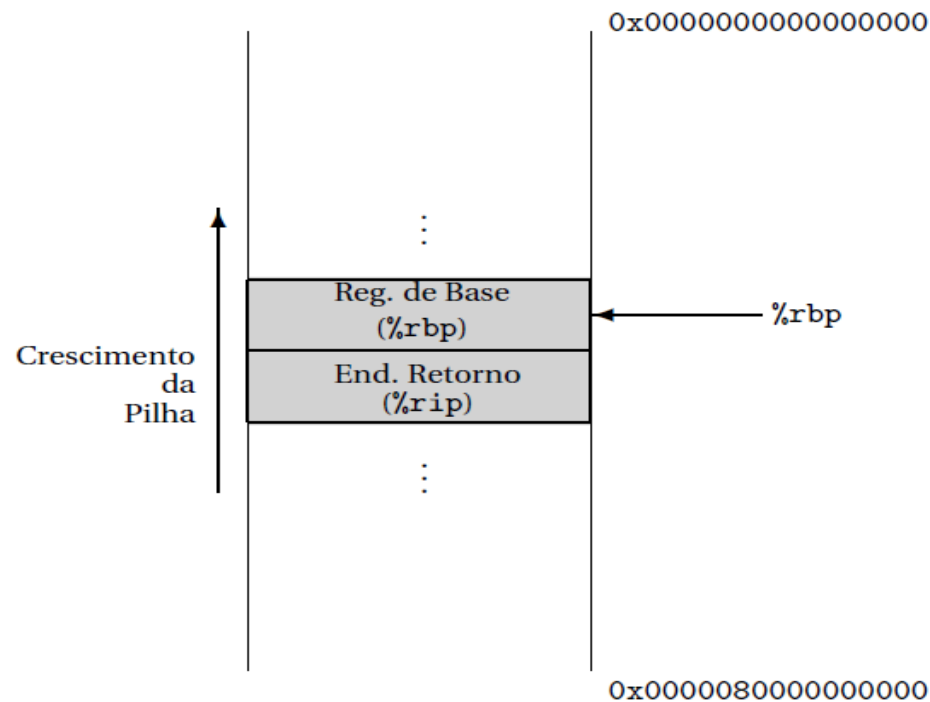
Linguagem Assembly

- **Sem Parâmetros e sem Variáveis Locais:**
 - Se eliminarmos o espaço destinado a variáveis locais e a parâmetros, o que sobra é o espaço destinado às **informações de contexto**.



Linguagem Assembly

- **Sem Parâmetros e sem Variáveis Locais:**
 - o registro de ativação armazena duas informações:
o endereço de retorno e o registrador de base.



Linguagem Assembly

- **Sem Parâmetros e sem Variáveis Locais:**
 - Funcionalidade das instruções **pushq** e **popq**

pushq <parâmetro>	subq \$8, %rsp movq <parâmetro>, (%rsp)
popq <parâmetro>	movq (%rsp), <parâmetro> addq \$8, %rsp

- O valor anterior de %rbp é armazenado na pilha. Após armazená-lo, muda-se o valor de %rbp para que ele contenha o endereço onde está armazenado o valor anterior.
- O endereço de retorno é o endereço da instrução seguinte à chamada do procedimento. Este endereço está armazenado no registrador %rip.



Linguagem Assembly

- **Sem Parâmetros e sem Variáveis Locais:**

```
pushq $1      # empilha a constante 1
pushq $A      # empilha o endereço do rótulo A
pushq A       # empilha o conteúdo do rótulo A
pushq %rax    # empilha o conteúdo do registrador %rax
popq $1       # ERRO! O destino não é um local de escrita
popq $A       # ERRO! O destino não é um local de escrita
popq A        # desempilha o topo da pilha dentro do rótulo A
popq %rax     # desempilha o topo da pilha no registrador %rax
```



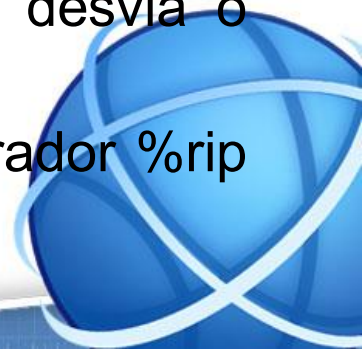
Linguagem Assembly

- **Sem Parâmetros e sem Variáveis Locais:**

- as instruções **pushq** (linha 7) e **popq** (linha 13) são utilizadas para salvar e restaurar o valor de **%rbp** utilizando a pilha.
- instruções é **call** e **ret** são utilizadas para salvar e restaurar o registrador **%rip** (*instruction pointer*) como parâmetro implícito.

<code>call <rotulo></code>	<code>pushq %rip</code> <code>jmp <rotulo></code>
<code>ret</code>	<code>popq %rip</code>

- A instrução **call** <rotulo> primeiro empilha o valor de **%rip** (ou seja, o endereço da próxima instrução) e depois desvia o fluxo para o rótulo indicado.
- Por outro lado, a instrução **ret** armazena no registrador **%rip** o valor contido no topo da pilha.



Linguagem Assembly

- Sem Parâmetros e sem Variáveis Locais:

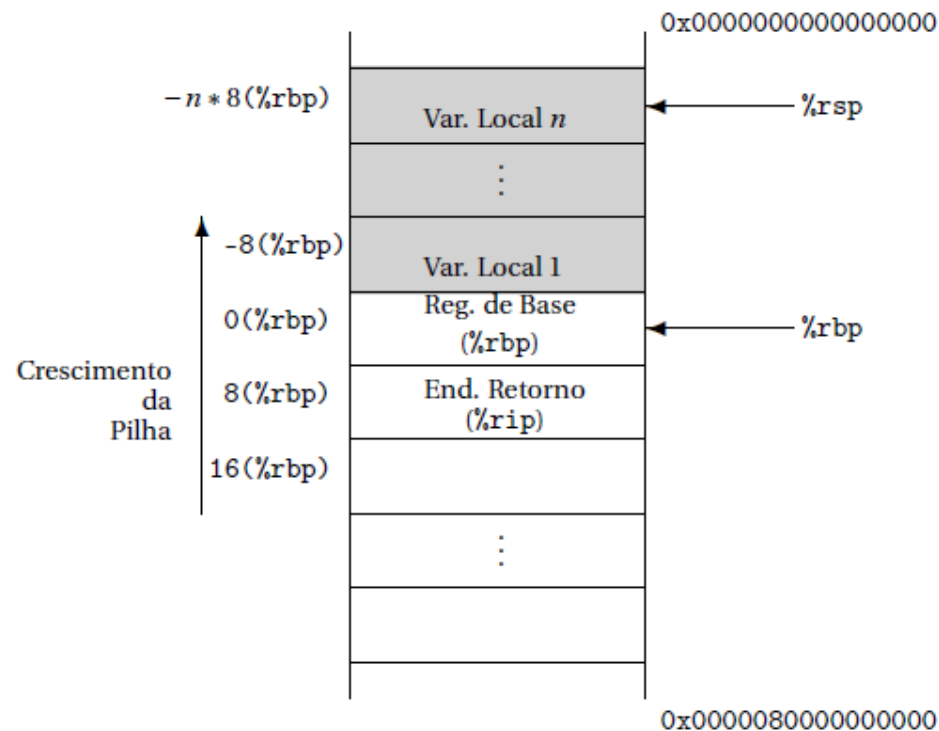
```
1 long int a, b;
2 int soma ( )
3 {
4     return (a+b);
5 }
6 int main (long int argc, char **argv)
7 {
8     a=4;
9     b=5;
10    b = soma();
11    return b;
12 }
```

```
1 .section .data
2 A: .quad 0
3 B: .quad 0
4 .section .text
5 .globl _start
6 soma:
7     pushq %rbp
8     movq %rsp, %rbp
9     movq A, %rax
10    movq B, %rbx
11    addq %rax, %rbx
12    movq %rbx, %rax
13    popq %rbp
14    ret
15 _start:
16    movq $4, A
17    movq $5, B
18    call soma
19    movq %rax, %rbx
20    movq $60, %rax
21    movq %rbx, %rdi
22    syscall
```



Linguagem Assembly

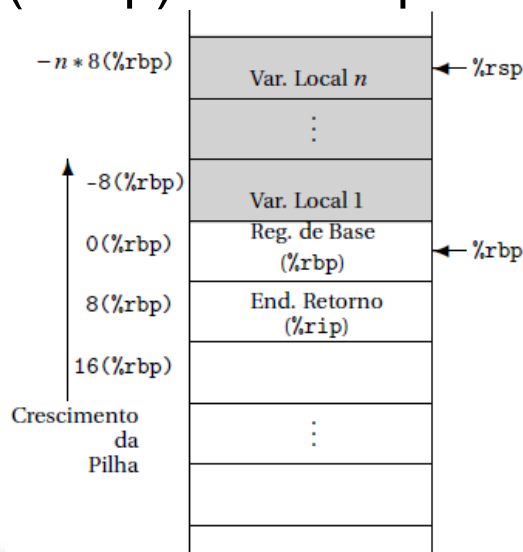
- **Sem Parâmetros e com Variáveis Locais:**
 - O acesso a variáveis locais e parâmetros de um registro de ativação se dá através do uso de um registrador que sempre aponta para o mesmo local do registro de ativação.



Linguagem Assembly

- **Sem Parâmetros e com Variáveis Locais:**

- As variáveis locais ficam localizadas imediatamente acima do local apontado por `%rbp`.
- Como ele sempre aponta para o mesmo local da moldura do registro de ativação, é possível determinar o endereço de cada variável: a primeira variável localiza-se em $-8(\%rbp)$. A segunda em $-16(\%rbp)$ e assim por diante.



Linguagem Assembly

- **Sem Parâmetros e com Variáveis Locais:**
 - Observe o sinal de menos à frente dos números. Como a pilha cresce para cima, é natural pensar que o sinal deveria ser positivo e não negativo. Porém, a pilha cresce para cima em direção aos endereços menores, ou seja de 0x0000080000000000 para 0x0000000000000000.
 - Este modo de determinar o endereço de variáveis é diferente do adotado para as variáveis globais na seção .data, onde o endereço de cada variável é fixo.



Linguagem Assembly

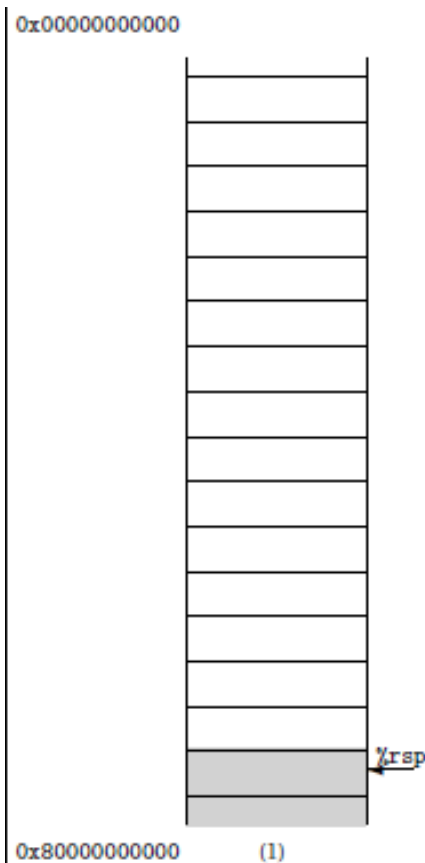
- Sem Parâmetros e com Variáveis Locais:

```
1 long int a, b;
2 int soma ( )
3 {
4     int x, y;
5     x=a;
6     y=b;
7     return (x+y);
8 }
9 int main (long int argc, char **argv)
10 {
11     a=4;
12     b=5;
13     b = soma();
14     return (b);
15 }
```

```
1 .section .data
2 A: .quad 0
3 B: .quad 0
4 .section .text
5 .globl _start
6 soma:
7     pushq %rbp
8     movq %rsp, %rbp
9     subq $16, %rsp
10    movq A, %rax
11    movq %rax, -8(%rbp)
12    movq B, %rax
13    movq %rax, -16(%rbp)
14    movq -8(%rbp), %rax
15    movq -16(%rbp), %rbx
16    addq %rbx, %rax
17    addq $16, %rsp
18    popq %rbp
19    ret
20 _start:
21    movq $4, A
22    movq $5, B
23    movq A, %rax
24    call soma
25    movq %rax, %rdi
26    movq $60, %rax
27    syscall
```

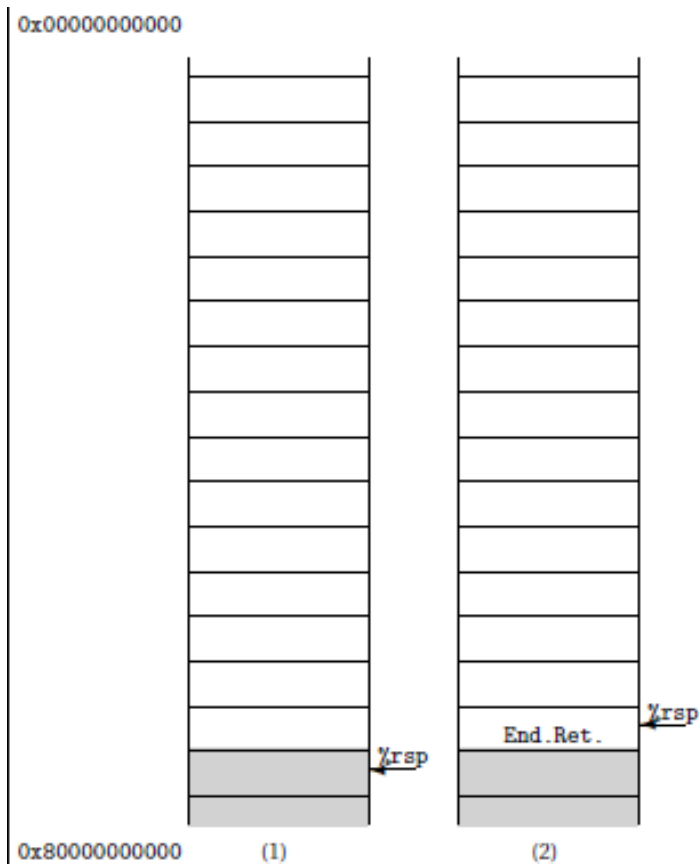
Linguagem Assembly

- Sem Parâmetros e com Variáveis Locais:



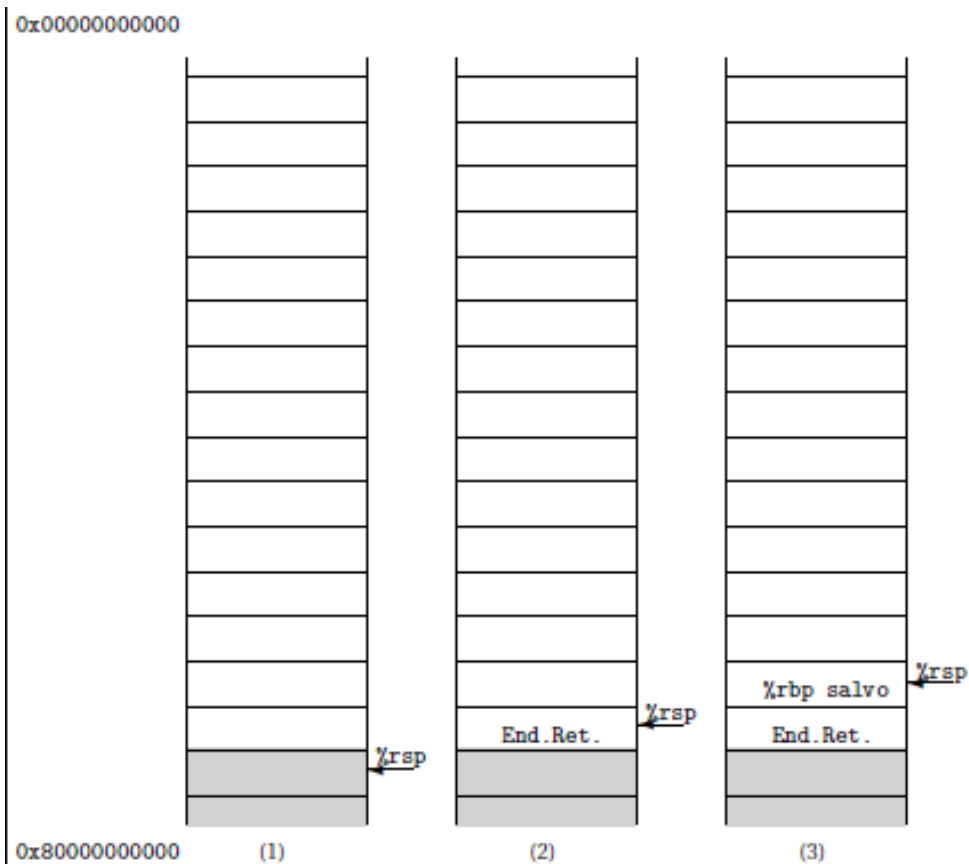
Linguagem Assembly

- Sem Parâmetros e com Variáveis Locais:



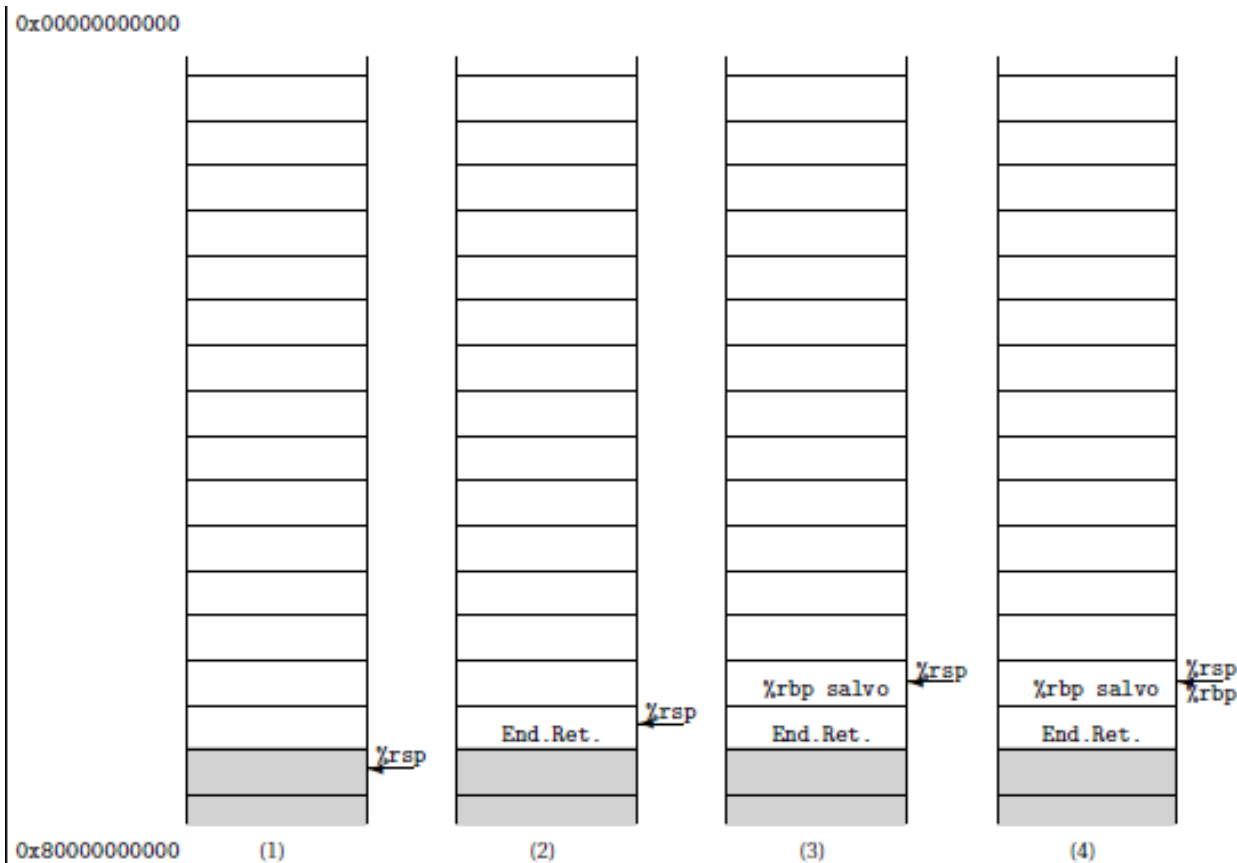
Linguagem Assembly

- Sem Parâmetros e com Variáveis Locais:



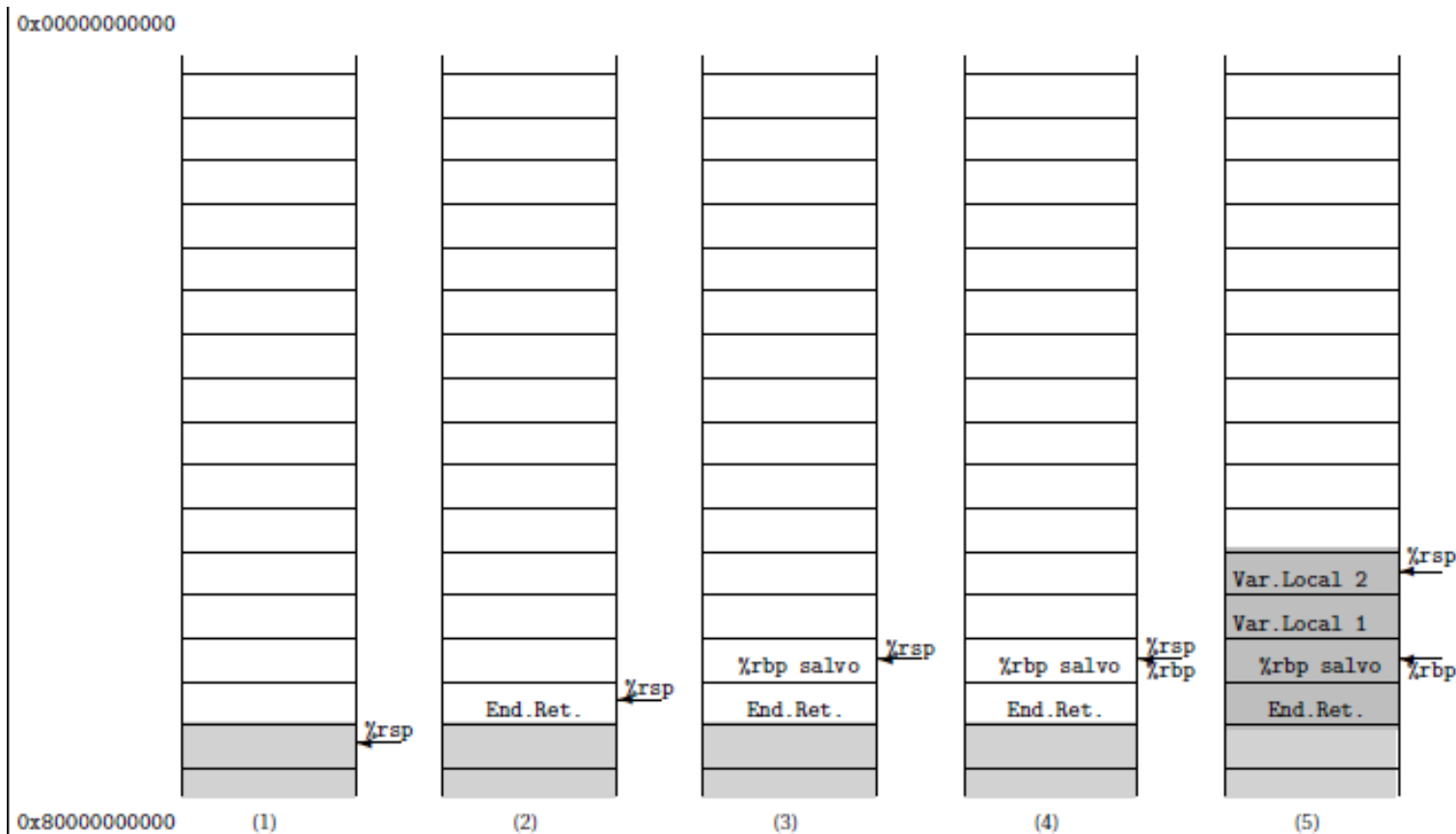
Linguagem Assembly

- Sem Parâmetros e com Variáveis Locais:



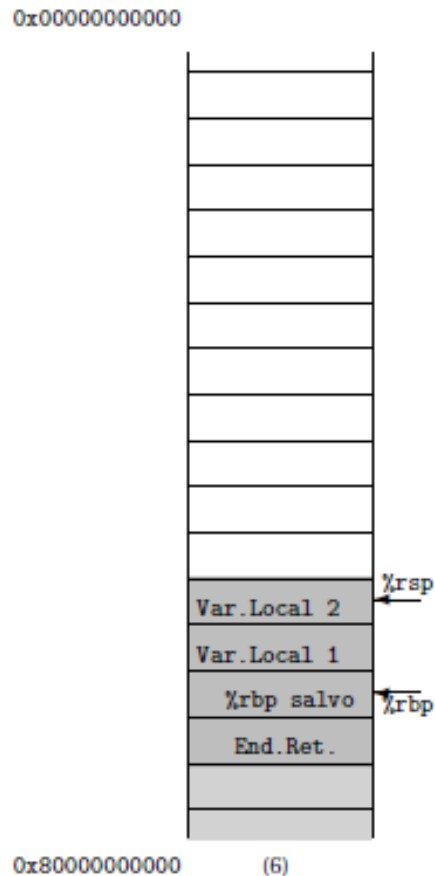
Linguagem Assembly

- Sem Parâmetros e com Variáveis Locais:



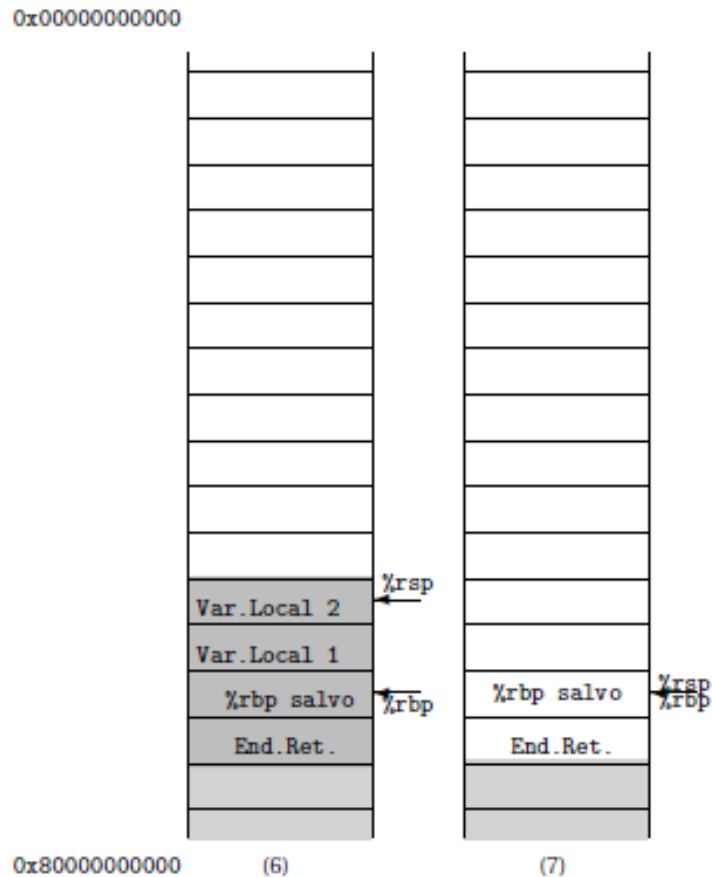
Linguagem Assembly

- Sem Parâmetros e com Variáveis Locais:



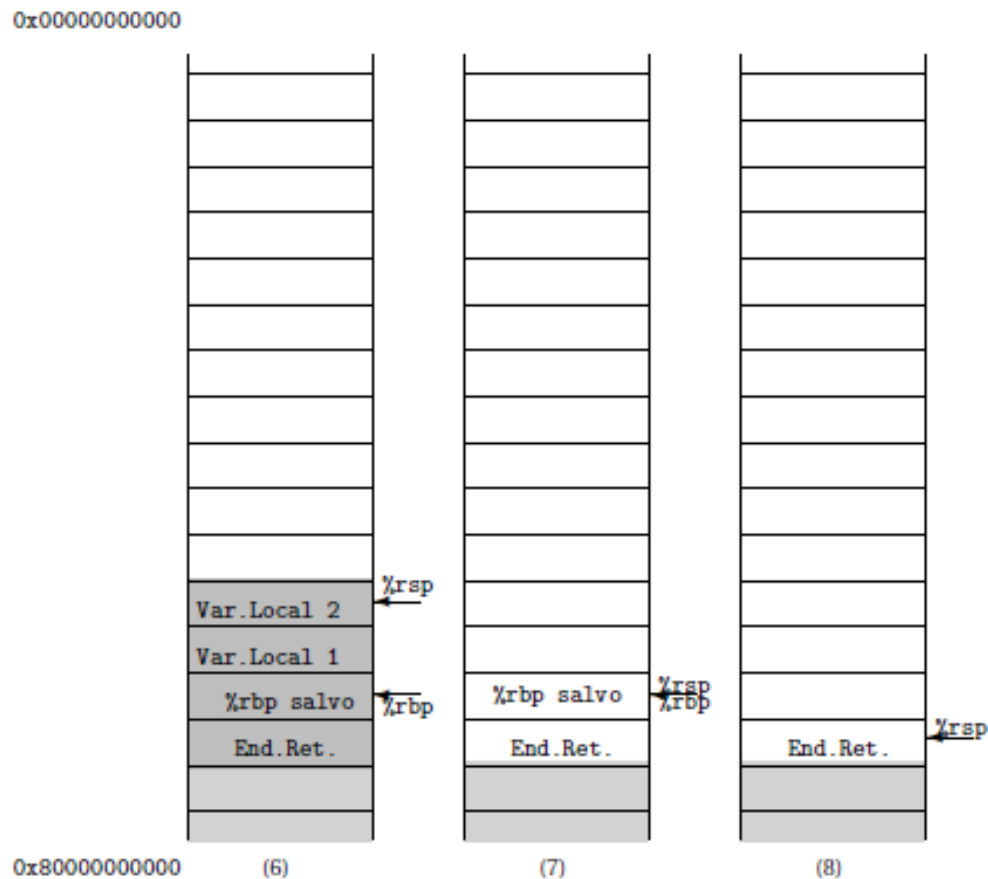
Linguagem Assembly

- Sem Parâmetros e com Variáveis Locais:



Linguagem Assembly

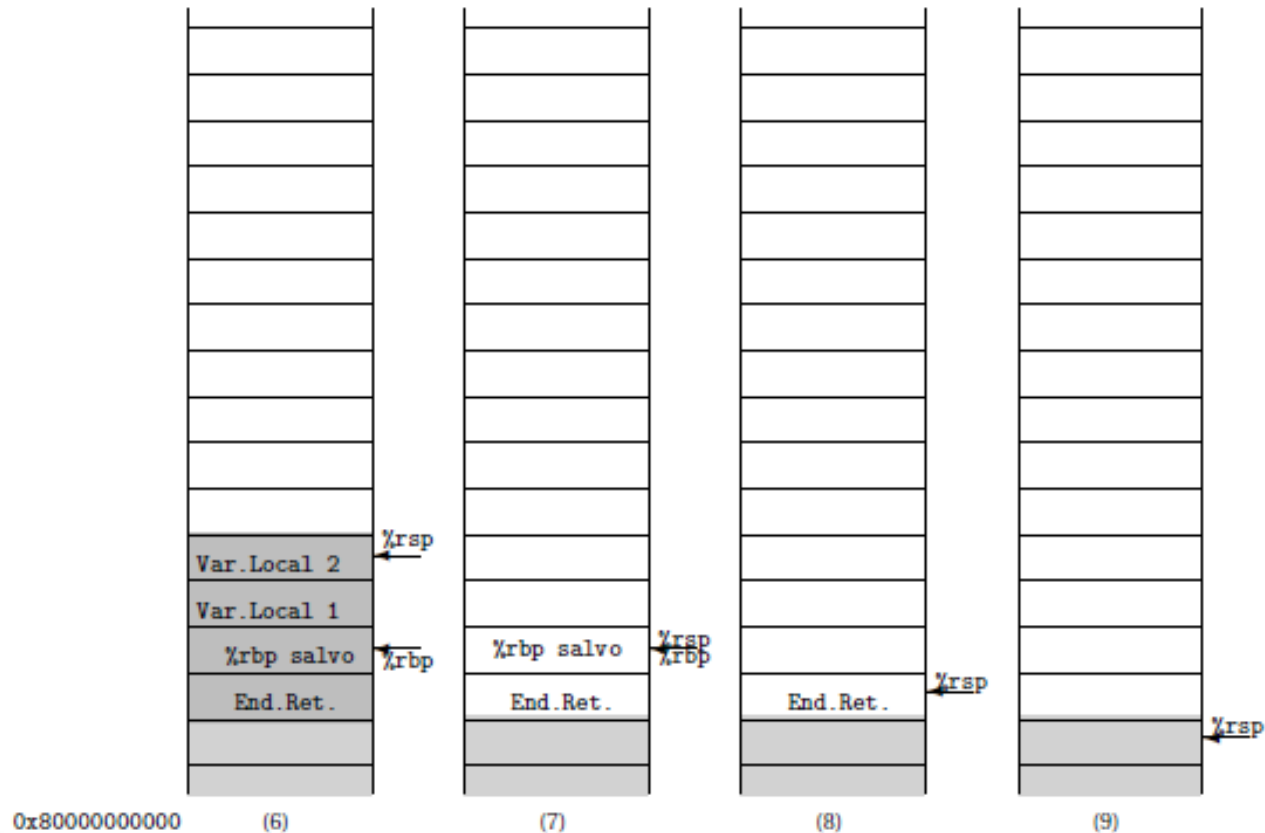
- Sem Parâmetros e com Variáveis Locais:



Linguagem Assembly

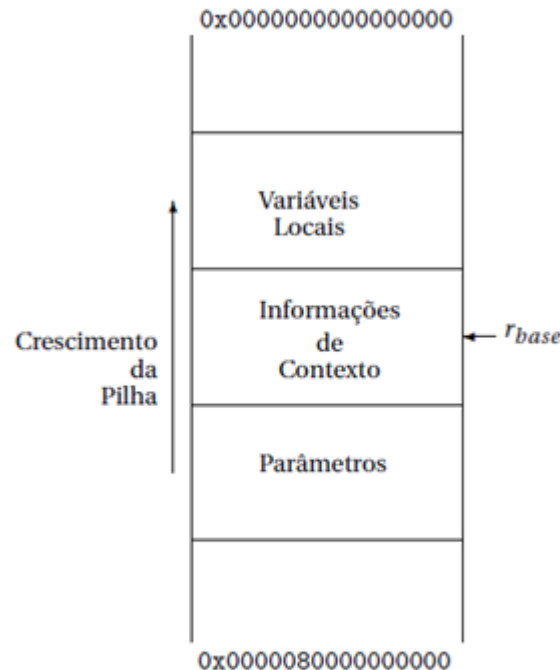
- Sem Parâmetros e com Variáveis Locais:

0x000000000000



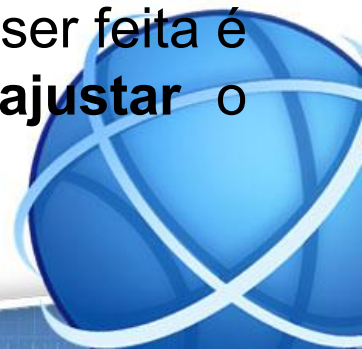
Linguagem Assembly

- **Com Parâmetros e com Variáveis Locais:**
 - Considere o procedimento P, que tem três parâmetro inteiros:
 - **P(long int a, long int b)**



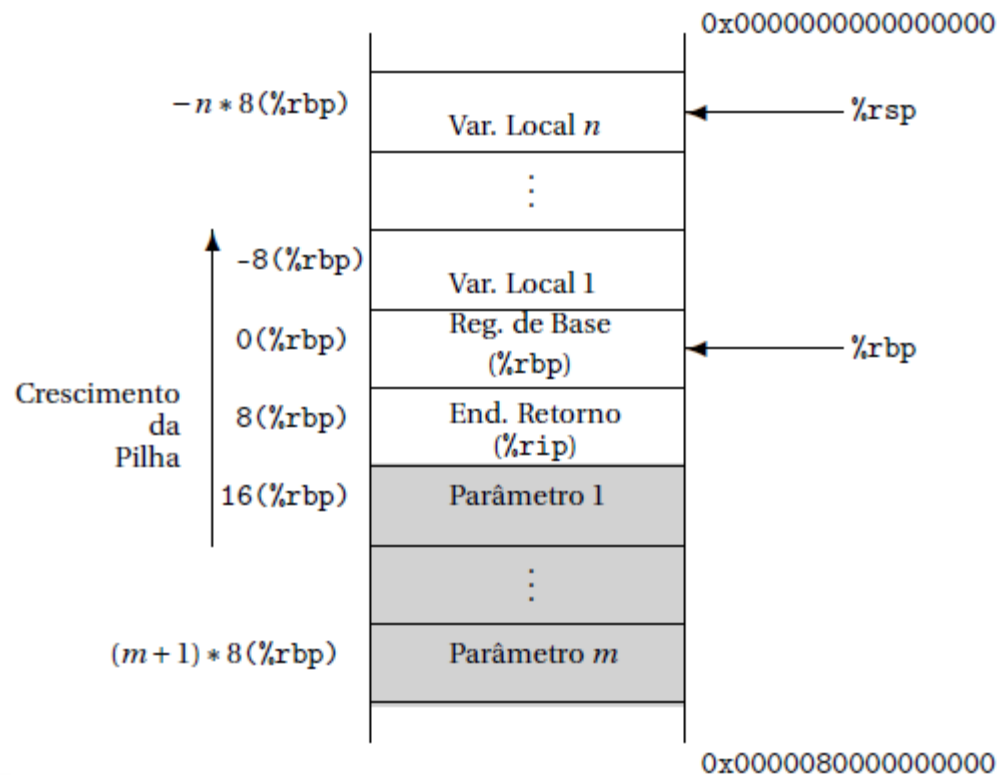
Linguagem Assembly

- **Com Parâmetros e com Variáveis Locais:**
 - Considere o procedimento P, que tem três parâmetro inteiros:
 - **P(long int a, long int b)**
 - **Os parâmetros** devem ser colocados na pilha **ANTES** das informações de contexto, ou seja, antes da instrução assembly **call**.
 - Ao retornar do procedimento, a altura da pilha será exatamente igual àquela que havia ao executar a instrução **call**, ou seja, com os parâmetros ainda empilhados.
 - Ao retornar do procedimento, a primeira coisa a ser feita é desempilhar os parâmetros. Para tal, basta **ajustar** o valor de **%rsp**.



Linguagem Assembly

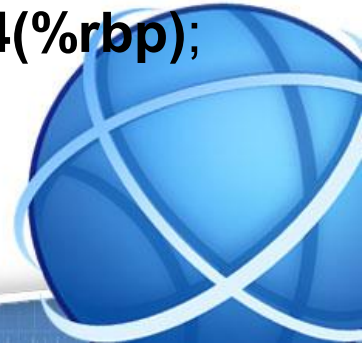
- **Com Parâmetros e com Variáveis Locais:**
 - Considere o procedimento P, que tem três parâmetro inteiros:
 - **P(long int a, long int b)**



Linguagem Assembly

- **Com Parâmetros e com Variáveis Locais:**

- Considere o procedimento P, que tem três parâmetro inteiros:
- **P(long int a, long int b)**
 - os parâmetros são empilhados na ordem inversa do programa C, ou seja, empilha b, empilha a;
 - após empilhá-los é que se faz a chamada ao procedimento;
 - no retorno do fluxo ao chamador, é que os parâmetros são desempilhados;
 - dentro do procedimento soma, as referências a **x** e **y** são traduzidas respectivamente como **16(%rbp)** e **24(%rbp)**;
 - a variável local z está localizada em **-8(%rbp)**.



Linguagem Assembly

- Com Parâmetros e com Variáveis Locais:

```
1 long int a, b;
2 int soma (long int x, long int y)
3 {
4     long int z;
5     z = x + y;
6     return (z);
7 }
8 int main (long int argc, char **argv)
9 {
10    a=4;
11    b=5;
12    b = soma(a, b);
13    return (b);
14 }
```

```
1 .section .data
2 A: .int 0
3 B: .int 0
4 .section .text
5 .globl _start
6 soma:
7     pushq %rbp
8     movq %rsp, %rbp
9     subq $8, %rsp
10    movq 16(%rbp), %rax
11    addq 24(%rbp), %rax
12    movq %rax, -8(%rbp)
13    addq $8, %rsp
14    popq %rbp
15    ret
16 _start:
17    movq $4, A
18    movq $5, B
19    pushq B
20    pushq A
21    call soma
22    addq $16, %rsp
23    movq %rax, %rdi
24    movq $60, %rax
25    syscall
```


Linguagem Assembly

- **Parâmetros por Referência e com Variáveis Locais:**
 - Os parâmetros da seção anterior foram passados por valor, ou seja, o valor foi copiado para a pilha. Isto faz com que a variável utilizada na chamada do procedimento não tenha o seu valor alterado quando do retorno.
 - A ideia básica é que o valor a ser empilhado na chamada não é uma cópia do valor a ser passado, porém uma cópia do **endereço da variável**.



Linguagem Assembly

- Parâmetros por Referência e com Variáveis Locais:

```
1 .section .data
2 A: .quad 0
3 B: .quad 0
4 .section .text
5 .globl _start
6 troca:
7     pushq %rbp
8     movq %rsp, %rbp
9     subq $8, %rsp
10    movq 16(%rbp), %rax
11    movq (%rax), %rbx
12    movq %rbx, -8(%rbp)
13    movq 24(%rbp), %rax
14    movq (%rax), %rbx
15    movq 16(%rbp), %rax
16    movq %rbx, (%rax)
17    movq -8(%rbp), %rbx
18    movq 24(%rbp), %rax
19    movq %rbx, (%rax)
20    addq $8, %rsp
21    popq %rbp
22    ret
23 _start:
24     movq $1, A
25     movq $2, B
26     pushq $B
27     pushq $A
28     call troca
29     addq $16, %rsp
30     movq $0, %rdi
31     movq $60, %rax
32    syscall
```



Linguagem Assembly: x86 vs AMD64

- **x86:**
 - Refere-se à arquitetura de processadores de 32 *bits*.
 - Originalmente desenvolvida pela Intel, a arquitetura x86 começou com o processador 8086 e evoluiu para incluir extensões como o 80386 (introduzindo suporte para proteção de memória e modo de 32 *bits*).
 - A arquitetura x86 é comumente referida quando se fala de processadores que operam em modo de 32 *bits*.



Linguagem Assembly: x86 vs AMD64

- **AMD64** (conhecida como x86-64 ou x64):
 - É uma extensão da arquitetura x86 que adiciona suporte para 64 *bits*.
 - Desenvolvida pela AMD e posteriormente adotada por outras fabricantes, como Intel (sob o nome **Intel 64**).
 - Introduz um conjunto de registradores adicionais, mais espaço de endereçamento de memória e melhorias na performance geral.



Linguagem Assembly: x86 vs AMD64

- Registadores x86 e AMD64

register encoding	high 8-bit	low 8-bit	16-bit	32-bit
0	AH (4)	AL	AX	EAX
3	BH (7)	BL	BX	EBX
1	CH (5)	CL	CX	ECX
2	DH (6)	DL	DX	EDX
6	SI		SI	ESI
7	DI		DI	EDI
5	BP		BP	EBP
4	SP		SP	ESP

31 16 15 0

Register Encoding	zero-extended for 32-bit operands		not modified for 16-bit operands		not modified for 8-bit operands		low 8 bits	16-bit	32-bit	64-bit
	32	31	16	15	8	7	0			
0					AH*		AL	AX	EAX	RAX
3					BH*		BL	BX	EBX	RBX
1					CH*		CL	CX	ECX	RCX
2					DH*		DL	DX	EDX	RDY
6							SIL**	SI	ESI	RSI
7							DIL**	DI	EDI	RDI
5							BPL**	BP	EBP	RBP
4							SPL**	SP	ESP	RSP
8							R8B	R8W	R8D	R8
9							R9B	R9W	R9D	R9
10							R10B	R10W	R10D	R10
11							R11B	R11W	R11D	R11
12							R12B	R12W	R12D	R12
13							R13B	R13W	R13D	R13
14							R14B	R14W	R14D	R14
15							R15B	R15W	R15D	R15

63 32 31 16 15 8 7 0

0										RFLAGS
										RIP

63 32 31 0

Linguagem Assembly: x86 vs AMD64

- **Endereçamento:**

- **x86:**

- Tem uma capacidade de endereçamento de memória limitada a 4 GB devido ao uso de endereços de 32 bits.

- **AMD64:**

- Suporta endereçamento de memória muito maior, teórica e praticamente suportando até 16 exabytes de memória.



Linguagem Assembly: x86 vs AMD64

- **Valores dos parâmetros de procedimentos passados em registradores:**
 - O tempo de acesso a informações armazenadas na memória é SUPERIOR ao tempo de acesso a dados em registradores da CPU.
 - Programas que maximizam o uso de registradores para armazenar variáveis apresentam um desempenho melhor do que programas equivalentes que não fazem essa otimização.
 - Existe uma convenção para o uso de registradores como local de armazenamento de parâmetros em chamadas de procedimentos.



Linguagem Assembly: x86 vs AMD64

- **Modelo de registro de ativação:**
 - **x86 - Passagem de Argumentos:**
 - Argumentos de procedimentos geralmente são passados na pilha. O chamador empurra os argumentos para a pilha antes da chamada.



Linguagem Assembly: x86 vs AMD64

- **Modelo de registro de ativação:**
 - **AMD64 - Passagem de Argumentos:**
 - Argumentos são passados usando registradores. Os primeiros seis argumentos inteiros ou ponteiros são passados em registradores (**%rdi, %rsi, %rdx, %rcx, %r8 e %r9**).
 - Argumentos adicionais são passados na pilha se houver mais do que seis.



Linguagem Assembly: Convenção de Chamada

- A convenção de chamada em AMD64 especifica o uso de registradores para argumentos e garantindo eficiência nas chamadas de procedimentos.

Argument Number	Argument Size			
	64-bits	32-bits	16-bits	8-bits
1	rdi	edi	di	dil
2	rsi	esi	si	sil
3	rdx	edx	dx	dl
4	rcx	ecx	cx	cl
5	r8	r8d	r8w	r8b
6	r9	r9d	r9w	r9b



Linguagem Assembly: Convenção de Chamada

- Essa convenção é utilizada por todas as bibliotecas da linguagem C no Linux, permitindo que programas escritos em Assembly sejam integrados a essas bibliotecas ao seguir a mesma convenção.
- Tanto **parâmetros** quanto **variáveis locais** devem ser alocados preferencialmente em **registradores** para **minimizar o acesso à memória** e consequentemente melhorar o desempenho dos programas;



Linguagem Assembly: Convenção de Chamada

- Alocar **argumentos** e **variáveis** em registradores entre chamadas aninhadas pode causar problema?



Linguagem Assembly: Convenção de Chamada

- Alocar **argumentos** e **variáveis** em registradores entre chamadas aninhadas pode causar problema?
- **Conflito de Registradores:** Registradores utilizados em uma chamada de função podem ser sobrescritos por uma chamada aninhada, o que pode levar à perda de dados se não forem corretamente salvos e restaurados.



Linguagem Assembly: Convenção de Chamada

- Para contornar este **problema**, a solução é salvar os valores contidos em **registradores na pilha**.
- Duas estratégias podem ser utilizadas:
 - **caller-save**: O chamador salva e restaura os registradores. Usado quando o chamador precisa garantir que os registradores que usa sejam preservados.
 - **callee-save**: O procedimento chamado salva e restaura os registradores. Usado quando a função ou procedimento precisa garantir que os registradores que usa não afetem o estado do chamador.



Linguagem Assembly: Convenção de Chamada

- Duas estratégias podem ser utilizadas:
 - Cada abordagem tem suas vantagens e desvantagens, e a escolha entre elas depende do modelo de programação e do ambiente em que você está trabalhando.
 - Algumas convenções de chamadas adotam uma combinação de ambos os métodos, dependendo do tipo de registradores e da situação específica.



Linguagem Assembly: Convenção de Chamada

- Os registradores %rax, %rcx, %rdx, %rsi, %rdi, %r8, %r9, %r10 e %r11 são considerados "registradores voláteis" na **convenção de chamada AMD64**.
- Embora possam ser modificados pelas funções chamadas, desta forma o **GCC** pode optar por salvar e restaurar esses registradores antes das chamadas para garantir que o código funcione corretamente e preserve os valores necessários.



63	31	15	8	7	0	
%rax	%eax	%ax	%ah	%al		Return value
%rbx	%ebx	%bx	%bh	%bl		Callee saved
%rcx	%ecx	%cx	%ch	%cl		4th argument
%rdx	%edx	%dx	%dh	%dl		3rd argument
%rsi	%esi	%si		%sil		2nd argument
%rdi	%edi	%di		%dil		1st argument
%rbp	%ebp	%bp		%bpl		Callee saved
%rsp	%esp	%sp		%spl		Stack pointer
%r8	%r8d	%r8w		%r8b		5th argument
%r9	%r9d	%r9w		%r9b		6th argument
%r10	%r10d	%r10w		%r10b		Caller saved
%r11	%r11d	%r11w		%r11b		Caller saved
%r12	%r12d	%r12w		%r12b		Callee saved
%r13	%r13d	%r13w		%r13b		Callee saved
%r14	%r14d	%r14w		%r14b		Callee saved
%r15	%r15d	%r15w		%r15b		Callee saved



Arquitetura - Uma Visão Geral do x86-64

- A combinação do novo hardware fornecido pela Intel e AMD, torna o código **x86-64** substancialmente diferente daquele gerado para máquinas **IA32**. As principais características incluem:
- Ponteiros e inteiros longos têm 64 bits de comprimento. As operações aritméticas com inteiros suportam tipos de dados de 8, 16, 32 e 64 bits.
- O conjunto de registradores de uso geral foi expandido de 8 para 16 para valores inteiros.



Arquitetura - Uma Visão Geral do x86-64

- Os parâmetros são mantidos em registradores em vez da pilha. Argumentos de procedimentos inteiros e ponteiros (até 6) são passados por registradores.
- As operações de ponto flutuante são implementadas utilizando o conjunto de instruções orientado a registradores introduzido com o **SSE versão 2**, em vez da abordagem baseada em pilha usada pelo IA32.



Arquitetura - Uma Visão Geral do x86-64

C declaration	Intel data type	Assembly code suffix	x86-64 size (bytes)	IA32 Size
char	Byte	b	1	1
short	Word	w	2	2
int	Double word	l	4	4
long int	Quad word	q	8	4
long long int	Quad word	q	8	8
char *	Quad word	q	8	4
float	Single precision	s	4	4
double	Double precision	d	8	8
long double	Extended precision	t	10/16	10/12



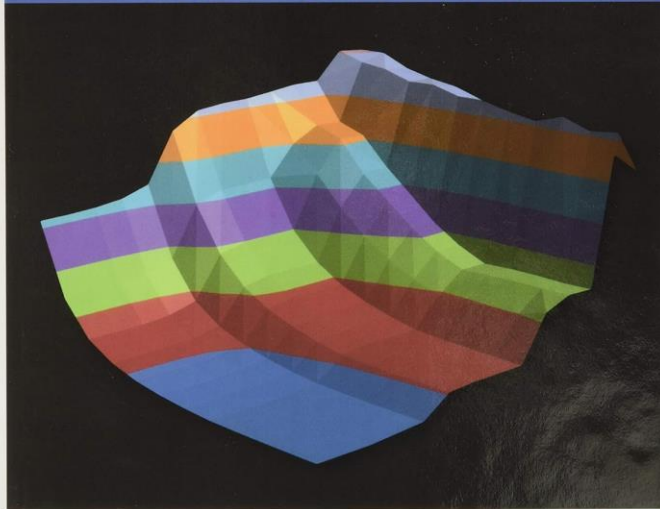
Dúvidas?



THIRD EDITION

COMPUTER SYSTEMS

A PROGRAMMER'S PERSPECTIVE



BRYANT • O'HALLARON

